

10/09/2025

Task-8.1(a) 7.1(a)

Aim: write a python program to implement python generator and decorators.

Algorithm:

- 1) Define generator function:
- 2) Initialize current value:
- 3) Generate sequence
- 4) Get user input.
- 5) Create generator object
- 6) print generated sequence

program:

```
def number-sequence
    current = start
    while current <= end
        yield current
        current += step
start = int(input("Enter the starting number:"))
end = int(input("Enter the ending number:"))
step = int(input())
# create the generator
sequence-generator = number-sequence
# print the generated sequence of numbers
for numbers in sequence-generator:
    print(number)
```

⇒ 8.1(b) 7.1(b)

Aim: produce a default sequence of numbers starting from 0, ending at 10, and with a step of 1 if no values are provided.

Algorithm:

- 1) start function
- 2) Initialize counter
- 3) Generate values
- 4) Create generator object:
- 5) Iterate and print

output:

Enter the starting Number = 1

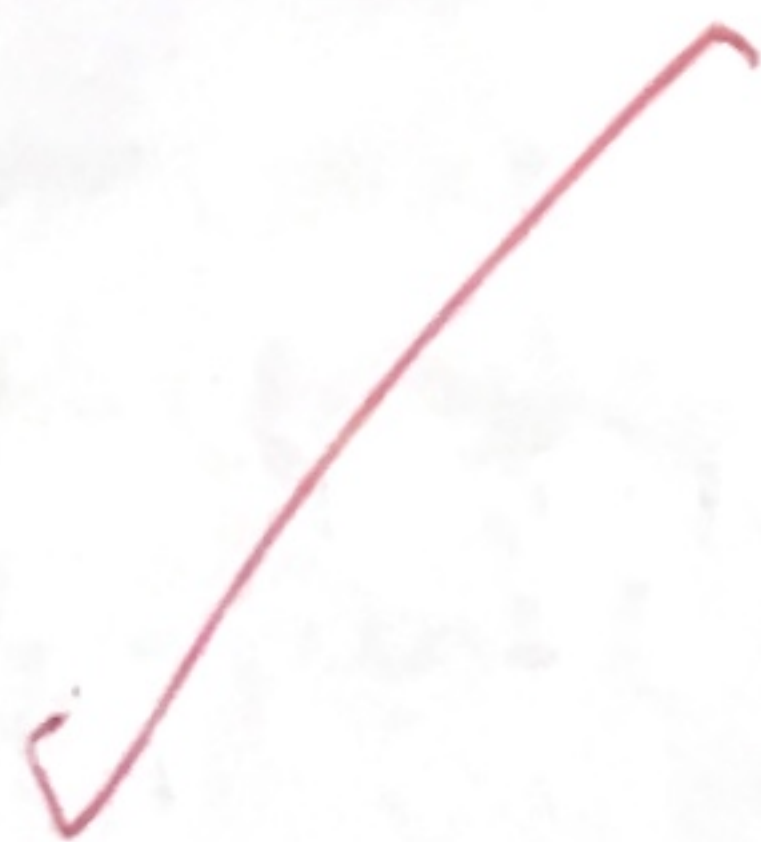
Enter the ending number = 50

Enter the step value: 5

1
6
11
16
21
26
31
36
41
46

program:

```
def my-generator(n):  
    # initialize counter  
    value = 0  
    # loop until counter value of the counter  
    yield value < n:  
    # produce the current value of the counter  
    yield value  
    # increment the counter  
    value += 1  
# iterate over the generator object produced by  
my_generator  
print(value)
```



every number is a part of a sequence (with a value) and
every number is a part of a sequence

Output:

- 0
- 1
- 2

(1) start function
(2) initialize current value
(3) generate sequence
(4) print the sequence
(5) create generator object
(6) print the generated sequence
(7) return

def number_generator():
 current = 0
 while current < 10:
 yield current
 current += 1

start = int(input("Enter the starting number"))
end = int(input("Enter the ending number"))
step = int(input())
create the generator
sequence_generator = number_generator()
print the generated sequence of numbers
for numbers in sequence_generator:
 print(numbers)

0 most primitive number to generate the sequence of numbers starting from 0
and value is 1 to 10 and it is a value of 10 and it is a value of 10 and it is a value of 10

(1) start function
(2) initialize current
(3) generate sequence
(4) create generator object
(5) print the sequence
(6) return

Task-8.2

Aim: It is to work on a messaging application that needs to format messages.

Algorithm

1) Create Decorators:

- Define uppercase-decorator.
- Define lowercase-decorator.

2) Define Functions:

3) Define Greet function

4) Execute the program:

program

```
def uppercase-decorator(func):
```

```
    def wrapper(text):
```

```
        return func(text).upper()
```

```
    return wrapper
```

```
def lowercase-decorator(func):
```

```
    def wrapper(text):
```

```
        return func(text).lower()
```

```
    return wrapper
```

```
@uppercase-decorator
```

```
def shout(text):
```

```
    return text
```

```
@lowercase-decorator
```

```
def whisper(text):
```

```
    return text
```

```
def greet(func):
```

```
    greeting = func("Hi, I am function passed as an  
argument.")
```

```
    greet(shout)
```

```
    greet(whisper)
```

VEL TECH	
EX NO.	7
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	
TOTAL (20)	
WITH DATE	15

Result: Thus, the python program to implement python generator and decorators was successfully executed and the output was verified.

! (1) rate ramp - 2 per 100

return value of the counter

0 = value

Output:

Hi, I AM CREATED BY A FUNCTION PASSED AS AN ARGUMENT.

Hi, i am created by a function passed as an argument.

return value

return all the counter

1 = 1 = value

Hi, I AM CREATED BY A FUNCTION PASSED AS AN ARGUMENT.

return value of the counter

(value) return

